

DISCODE

DIScover Coupled Ordinary Differential Equations

A complete technical report on the algorithm: the grammar, the work-energy reward, variable-projection constant fitting, the policy (masked-diffusion and the joint autoregressive replacement), the J-GRPO training loop, every configuration parameter, the known failure modes, and the measurements that support each claim.

Subject	Deep symbolic regression for equations of motion from response data
Codebase	hvnsl/DISCODE, branch claude/autoregressive-joint-policy-z6fjb1
Reward	Per-DOF work-energy (power) balance, optionally blended with acceleration NRMSE
Policy	Joint autoregressive transformer (default) or N independent masked-diffusion models
Optimiser	J-GRPO: PPO-clipped policy gradient with KL-to-reference, on a replay buffer
Scope of this report	Every module in the repository, as the code stands at the time of writing

How to read this document. Sections 2 to 8 follow the data as it moves through one epoch: raw channels become normalised trajectories, the grammar defines what an expression may be, the policy proposes token sequences, variable projection fits their constants, the work-energy reward scores them, and J-GRPO turns those scores into a gradient. Section 9 is a complete parameter reference. Sections 10 to 12 cover the ablation design, the failure modes that actually occur in practice, and the limitations that remain open. Every quantitative claim in this report was produced by running the code, and the numbers are reported as measured.

Contents

- 1 What DISCODE does, and the one idea it is built on
- 2 System architecture and module map
- 3 The data layer
- 4 The grammar: what an expression is allowed to be
- 5 Expression evaluation and the four-implementation problem
- 6 The reward: a per-DOF work-energy balance
- 7 Constant fitting by variable projection
- 8 The policy
- 9 The training loop and J-GRPO
- 10 Complete configuration reference
- 11 The ablation design
- 12 Failure modes and diagnostics
- 13 Limitations and open problems
- A Appendix A - worked example, end to end
- B Appendix B - function reference

A note on terminology. A *candidate* is one token sequence, written τ , that the policy has emitted. A *DOF* is one degree of freedom of the mechanical system; an N-DOF system has $2N$ state variables (a position and a velocity each) and needs N expressions, one per acceleration. A *trial* or *trajectory* is one recorded or simulated free-decay response; several are used at once so that a candidate must explain more than one initial condition.

1 What DISCODE does, and the one idea it is built on

DISCODE recovers the equations of motion of a mechanical system from measured or simulated response data. Given displacement, velocity and acceleration channels for each degree of freedom, it searches for a closed-form symbolic expression for each DOF's acceleration:

```
q1'' = f1(q1, q1dot, q2, q2dot, ...)
q2'' = f2(q1, q1dot, q2, q2dot, ...)
...
```

The search is a reinforcement-learning loop. A neural policy emits candidate expressions as token sequences; each candidate has its free constants fitted to the data; the fitted candidate is scored; and the score drives a policy-gradient update. This is the deep-symbolic-regression pattern, following Bastiani et al. (2025). What distinguishes DISCODE is the objective.

1.1 The work-energy reward

The obvious objective for this problem is pointwise acceleration error: compare the predicted acceleration against the measured one and minimise NRMSE. The other obvious choice is forward-simulation error: integrate the candidate and compare trajectories. DISCODE uses neither as its primary signal. It uses the **work-energy theorem**.

For a mass, the power delivered integrates to the change in kinetic energy. Because the discovered expression predicts acceleration directly, the mass cancels:

$$\begin{aligned} d/dt \left(\frac{1}{2} m v^2 \right) &= v * F = v * m * a \\ \Rightarrow \int_{0..t} v(s) a(s) ds &= \frac{1}{2} (v(t)^2 - v(0)^2) \end{aligned}$$

The right-hand side is measured. The left-hand side is computed by putting the *candidate's* predicted acceleration under the integral, evaluated on the *measured* states. The gap between the two, accumulated over time, is the residual that the reward penalises.

Three consequences follow, and they shape everything else in the system:

- **No forward integration is ever required.** The reward reads the measured velocity and the predicted acceleration off the same time grid and applies the trapezoid rule. A candidate that is numerically unstable, or that would blow up if integrated, still gets a score. This makes scoring cheap and robust — and, as Section 12.3 shows, it is also the single largest weakness of the objective.
- **The balance is exact per DOF and closes independently.** The identity above holds for each DOF on its own. Whatever coupling forces the other DOFs exert are already contained in that DOF's acceleration, so DOF *i*'s balance needs no knowledge of DOF *j*. This is why the reward, the constant fit and the replay buffers are all per-DOF, even under a joint policy.
- **The objective is integral, not pointwise.** Over a cycle the conservative terms do zero net work while dissipative terms accumulate. The energy objective therefore weights terms by dissipated work, which is a completely different weighting from pointwise error. Section 6.4 measures the difference.

The identity in code. Per DOF *d*, with velocity v_d and predicted acceleration a_d :

```
E_d(t) = | cumtrapz(v_d * a_d, t) - 1/2 ( v_d(t)^2 - v_d(0)^2 ) | / s_d
r = 1 / ( 1 + mean_over_trials[ mean_over_DOFs[ mean_over_t E_d(t) ] ] )
```

with $s_d = \text{std}(dKE_d)$ the per-DOF normaliser that stops a high-velocity DOF from swamping a low-velocity one. $r = 1$ is a perfect match.

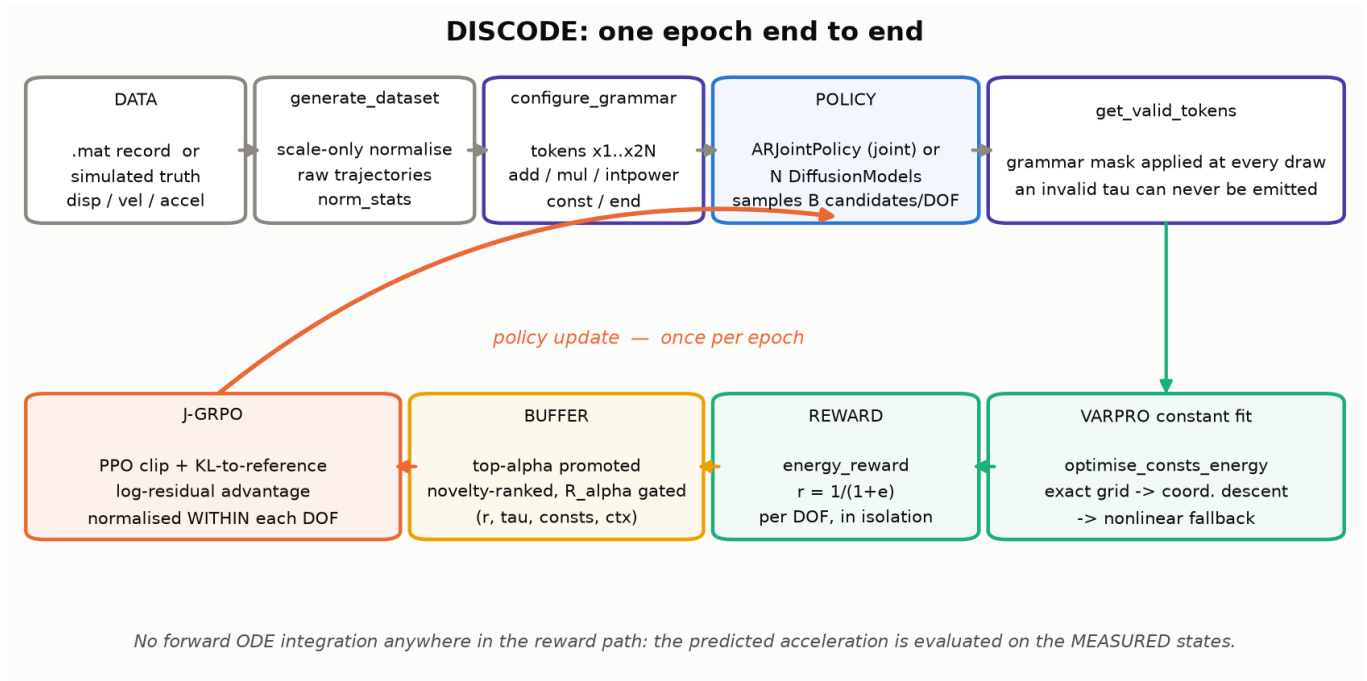


Figure 1. One epoch of DISCODE end to end. The loop is: sample candidates under a grammar mask, fit their constants by variable projection, score them against the work-energy balance, promote the best into a per-DOF replay buffer, and take a J-GRPO step. Nothing in the reward path integrates an ODE.

2 System architecture and module map

The repository separates concerns strictly: nothing in the engine knows where the data came from, and nothing in the drivers knows how the search works. The engine is configured by exactly two calls — `configure_grammar(n_dof)` once the DOF count is known, and `set_problem_data(...)` once trajectories exist.

Module	LOC	Responsibility
<code>discode_core.py</code>	1777	The engine. Grammar tables and validity masking, expression evaluation (four separate implementations), VARPRO constant fitting, the work-energy reward, the masked-diffusion policy and critic, J-GRPO, and the process-pool scoring worker.
<code>discode_policy.py</code>	~600	The joint autoregressive policy: model, attention masks, sampling, per-slice beam search, and the AR form of the J-GRPO objective.
<code>discode_data.py</code>	579	Everything that produces a <code>SystemData</code> : MATLAB loading (v7.3 and v5/v6), truth simulation, dataset normalisation, and the identity-ceiling diagnostic.
<code>discode_train.py</code>	~560	The single N-DOF training loop, the policy/DOF state split, buffer promotion, and the critic.
<code>discode_analysis.py</code>	456	Post-hoc only: expression parsing, forward simulation, energy-residual curves, comparison plots, and scoring against the training reward.
<code>discode_<sdof mdof>_sim exp>.py</code>	~150	Four drivers. The sim drivers carry a library of known systems; the exp drivers load a MATLAB record. All four build a <code>SystemData</code> and call <code>DISCODE_TRAIN</code> .
<code>discode_score.py</code> , <code>discode_plot*.py</code>	~120	Paste-in tools for discovered expressions: reproduce the training reward, or forward-simulate and plot against the record.
<code>discode_policy_test.py</code>	~330	Executable correctness invariants for the autoregressive policy. 27 checks.

A single-DOF run is just $N = 1$. There is no separate SDOF path anywhere: the same loop, the same grammar machinery and the same reward handle one DOF and many. The drivers differ only in how they build the `SystemData`.

3 The data layer

3.1 SystemData

Everything enters the pipeline as a `SystemData`: three arrays of shape $(m, n_{\text{dof}}, p_{\text{trials}})$ for displacement, velocity and acceleration on a common time grid, plus per-DOF display names and, for simulated systems, the ground truth.

```
SystemData(name, acc, vel, disp, time, var_names, truth_strs, truth_taus)

acc, vel, disp : (m, n_dof, p_trials)
time           : (m,)
var_names      : ['x', 'y'] -> grammar sees x, xdot, y, ydot
truth_strs     : human-readable ground truth, or 'unknown'
truth_taus     : ground-truth token sequences, or None
```

The state layout is fixed and interleaved. For an N-DOF system there are $2N$ grammar variables: $x_1 = q_1$, $x_2 = \dot{q}_1$, $x_3 = q_2$, $x_4 = \dot{q}_2$, and so on. Getting this ordering wrong silently produces plausible nonsense, so it is worth stating twice: **odd indices are positions, even indices are velocities.**

3.2 Simulated systems

A `TruthSystem` is a declarative spec: a list of acceleration functions, an initial-condition amplitude vector, a time grid, a trial count and a seed. Initial conditions are drawn uniformly from $[-ic_scale, +ic_scale]$ with that fixed seed, so a given spec always reproduces the same dataset — which is what makes the plotters able to reconstruct exactly the data a run was trained on.

Two spec fields deserve care. `ic_scale` must be set to the amplitude at which the nonlinearity actually lives: too small and a cubic term is unidentifiable from the data, too large and the response leaves the regime of interest. `n_pts` must resolve the $v \cdot a$ integrand, which has content up to twice the highest natural frequency — Section 3.5 quantifies what happens when it does not.

3.3 Experimental loading

`load_mat_data` handles both MATLAB v7.3 (HDF5, via `h5py`, transposing because MATLAB is column-major) and v5/v6 (via `scipy.io.loadmat`, which already returns the right shape). It then trims a configurable number of samples from the front and back — to keep only the free-decay portion — and downsamples to roughly `desired_timesteps`.

The trim and downsample settings are part of the problem definition, not a display preference. They change the time grid the equations are fitted on, so any later scoring or plotting must use identical settings or it is scoring a different problem. This is the most common practical mistake when using the analysis tools.

3.4 Normalisation is scale-only by default

`generate_dataset` normalises features and targets, but with `center=False` it divides by the standard deviation without subtracting the mean. This is deliberate and load-bearing.

Subtracting the mean is an affine change of variable, and a polynomial that is simple in physical coordinates is not simple in shifted ones. With $x = X_mean + X_std * u$, a physical term such as $\alpha \cdot (q_1 - q_2)^3$ expands into degree-3, degree-2, degree-1 *and* constant monomials in u . The grammar would then have to discover those extra terms merely to express the truth, which lengthens the target expression and hands a reward advantage to candidates that pad themselves with spurious low-order terms.

The code documents the measurement: on a cubic-coupled truth, the degree- $\{1,3\}$ expanded cube scores **0.99916 centred** versus **0.99998 uncentred** — a roughly 40x residual penalty paid purely for the offset. That gap is invisible in r -units but is exactly what the log-residual GRPO advantage of Section 9.4 chases. These are zero-mean free-decay signals about a homogeneous equilibrium, so there is nothing to centre in the first place.

3.5 The identity ceiling: the most important diagnostic in the system

Before any reward can be interpreted, one question must be answered: *how well does the measured acceleration itself satisfy the work-energy balance?* `identity_ceiling` computes exactly the training reward, but with the measured acceleration substituted for the predicted one. No expression can score above that number.

```
lhs = cumtrapz( v * a_measured, t )
rhs = 0.5 * ( v^2 - v[0]^2 )
r    = 1 / ( 1 + mean| lhs - rhs | / std(rhs) )    <-- the ceiling
```

The ceiling fails for two entirely different reasons, and telling them apart decides what to do next.

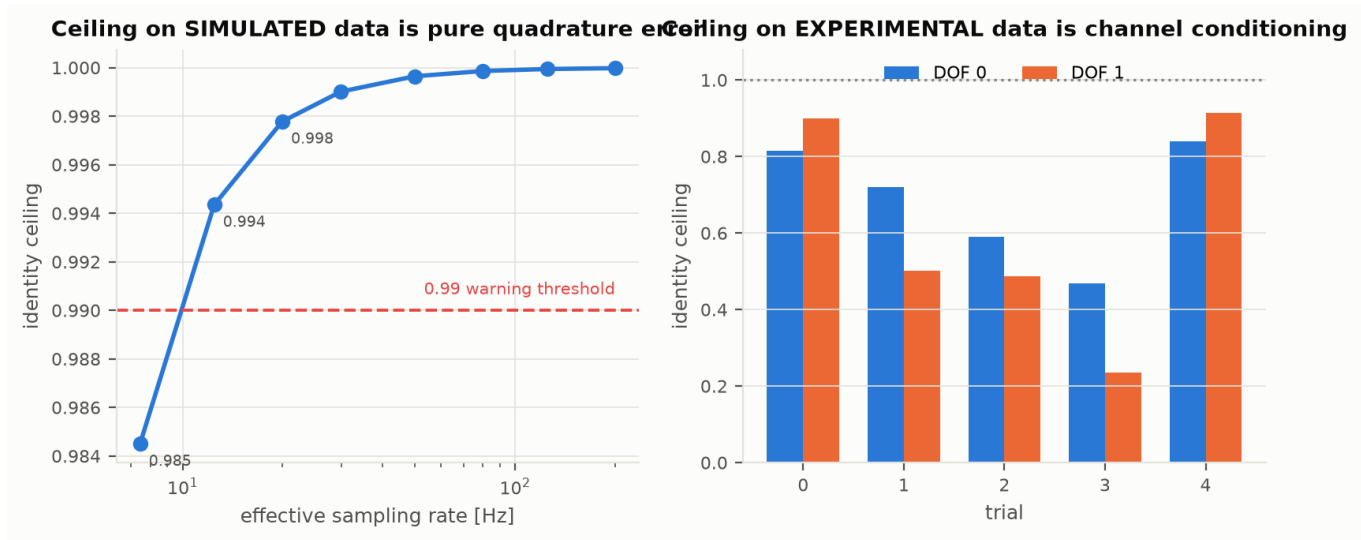


Figure 2. Left: on simulated data the states are exact by construction, so a ceiling below 1.0 can ONLY be trapezoid under-resolution of the $v \cdot a$ integrand. Measured on the SDOF Duffing system: 7.5 Hz gives 0.9845, and the ceiling clears 0.99 at roughly 12 Hz and 0.9999 by 125 Hz. The fix is more points, not a better search. Right: on experimental data the ceiling reflects channel conditioning. This is a real record measured during this work; trials 0 and 4 are usable at 0.81-0.91, while trial 3 reaches only 0.2354 on DOF 1. No amount of searching can recover that.

On simulated data the states are exact, so any shortfall is quadrature error. The $v \cdot a$ integrand needs roughly 40 points per cycle of the fastest mode; `identity_ceiling(warn=True)` prints an explicit warning below 0.99.

On experimental data the ceiling reflects whether the disp / vel / acc channels are mutually consistent. Anything applied per integration stage — a high-pass to kill drift, a band-pass on the accelerometer — breaks $a = dv/dt$ and $v = dq/dt$, and the identity degrades. The fix is in the data conditioning: **every channel must be filtered with the same number of passes**, or the derivative relations between them no longer hold.

Read the ceiling before reading any reward. Near a low ceiling the ranking between candidate expressions can invert, which means the search is optimising measurement artefacts. A run against a ceiling of 0.24 does not measure the dynamics; it measures the filtering. Section 12 shows what the resulting expressions look like.

4 The grammar: what an expression is allowed to be

An expression is a tree, and the policy emits it as a flat **pre-order token sequence** called τ . The grammar decides which token may follow any given prefix. Because that decision is enforced at every single draw, an ungrammatical τ is not merely penalised — it cannot be emitted at all.

4.1 The token set

`configure_grammar(n_dof)` builds the table. For an N-DOF system:

```
ALL_TOKENS = BINARY_OPS + UNARY_OPS + VARIABLES + CONSTANTS + ['end']
            = ['add', 'mul'] + ['intpower'] + ['x1'..'x2N'] + ['const'] + ['end']

N_TOKENS   = 2N + 5          # N=2 -> 9 tokens
MASK_TOKEN = N_TOKENS       # the extra input-only symbol
VOCAB_SIZE = N_TOKENS + 1    # embedding table size
```

Token	Arity	Constant slots	Meaning
add	n-ary	0	Sum of two or more children; closed by end
mul	n-ary	0	Product of two or more children; closed by end
intpower	1	2 (coefficient, exponent)	$c * u^n$ with n a fitted, snapped integer
x1 .. x2N	0	1 (coefficient)	State variable; odd = position, even = velocity
const	0	1 (value)	Free additive constant
end	0	0	Closes the innermost n-ary node

Two design choices in that table carry most of the grammar's expressive economy. **Every variable leaf carries an implicit fitted coefficient**, so `['add', 'x1', 'x2', 'end']` already means $c_1 q_1 + c_2 q_1 \dot{}$ — no explicit multiply is needed for a linear combination, and **signs come out of the fit for free**. And **add and mul are n-ary**, so a six-term polynomial is one add node rather than five nested binary ones.

Two power operators are present in the code but disabled. `abspower` ($c * |u|^p$) and `sgnpower` ($c * \text{sign}(u) * |u|^p$) take continuous exponents in `[MIN_EXP, MAX_EXP] = [0.5, 7.0]`. They are re-enabled by setting `POWER_OPS = ('abspower', 'sgnpower', 'intpower')`. The trailing comma in the current one-element tuple is load-bearing: `('intpower')` is the string, not a tuple, and would silently break every membership test.

4.2 Pre-order encoding and constant slots

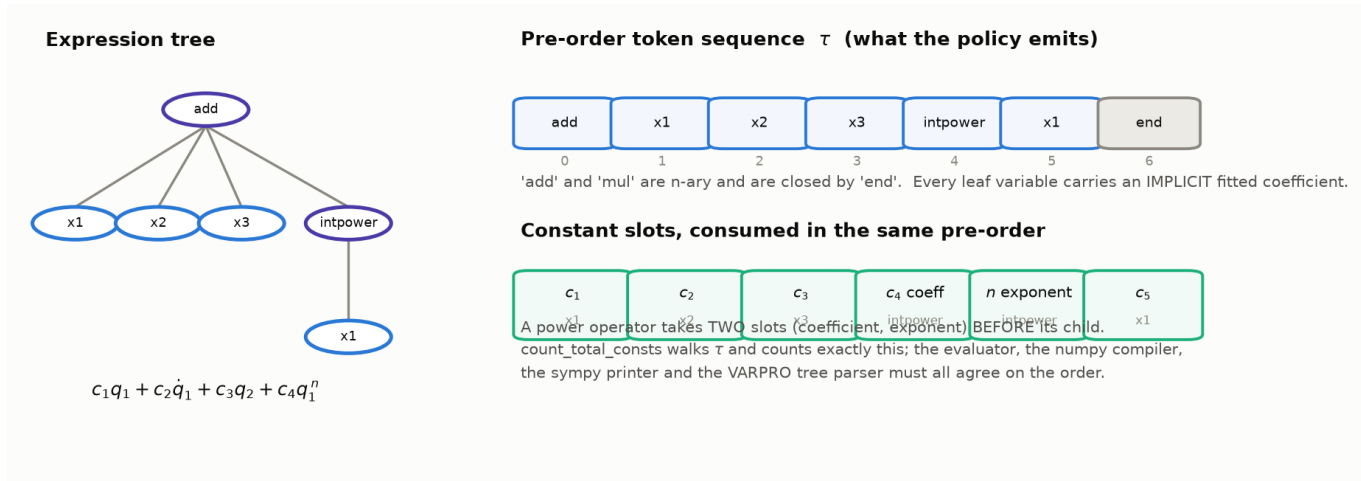


Figure 3. The same expression in three representations. The policy emits only the middle one. Constant slots are assigned in the same pre-order the evaluator consumes them, and a power operator takes TWO slots — coefficient then exponent — BEFORE its child. Four separate pieces of code must agree on this ordering.

`count_total_consts(tau)` walks the sequence and counts: one slot for each variable or const, two for each power operator. Nothing else consumes a slot. The constants themselves are never emitted by the policy — they are fitted numerically (Section 7), which is what keeps the discrete search space small.

4.3 Validity masking

`get_valid_tokens(tau, position, max_len)` returns a binary mask over the token table. It is the heart of the grammar, and it is called before every single token draw. It works by parsing the prefix into a stack of open obligations (`_parse_stack`) and then testing each candidate token against six rules.

Rule	What it enforces
Budget	A token is rejected if committing it would leave more open slots than there are positions remaining. This is what guarantees an expression can always be closed within <code>max_len</code> , and it is computed from remaining = max_len - position — which is why sampling must pass the intra-slice position, never a flattened index.
No leading leaf	At position 0 a zero-arity token is rejected, so an expression cannot be a bare variable.
Minimum length	A leaf that would complete the tree before <code>MIN_EXPR_LEN = 6</code> tokens is rejected, and end is likewise blocked if it would close the whole expression too early. This suppresses trivial one-term answers.
Depth	An operator is rejected once the current stack depth reaches <code>MAX_TREE_DEPTH = 4</code> . Look-ahead is included: an operator is also rejected if its child could be neither a legal leaf nor a legal deeper operator.
Constant cap	const is rejected once the prefix already holds <code>max_consts = 20</code> of them.
Forbidden nesting	add may not contain add; mul may not contain mul (both would be flattenable duplicates of the n-ary form); and a power operator may not contain a power operator, because $(u ^p)^q$ is degenerate with $ u ^{(pq)}$.

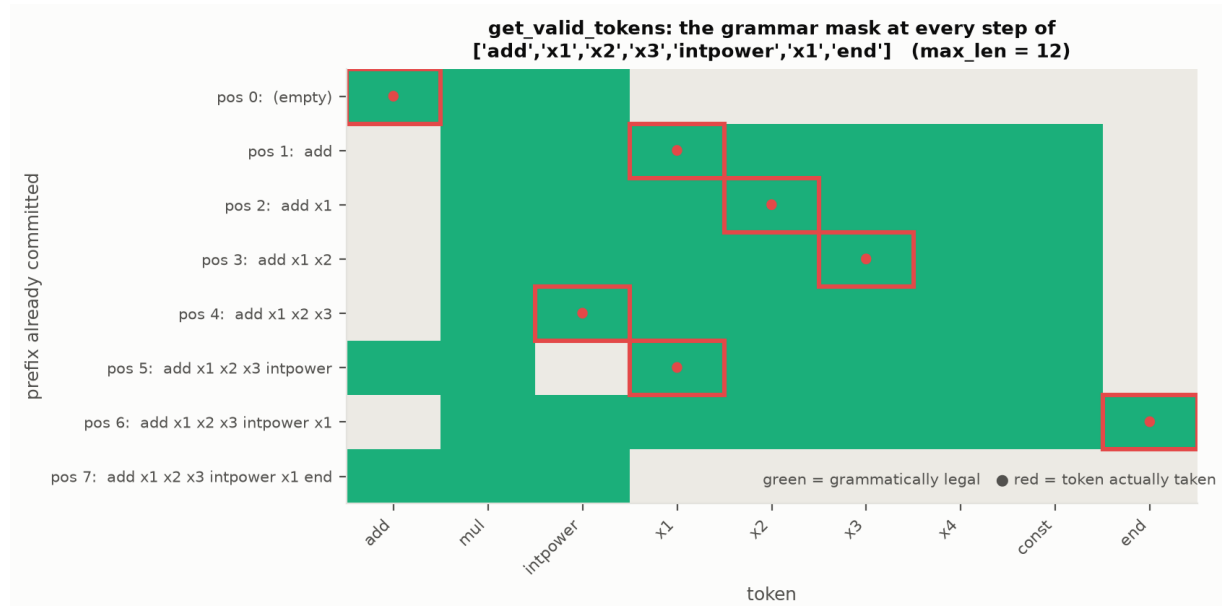


Figure 4. The mask actually returned by `get_valid_tokens` at every step of a real expression, generated by calling the function. At position 0 only the three operators are legal (no leading leaf). Once inside the `add` node, seven tokens are legal. At position 6 the budget has tightened and `end` becomes available. Valid-token counts across the sequence: 3, 7, 7, 7, 7, 7, 8, 3.

`is_complete(tau)` is the companion test: it replays the same stack discipline and reports whether every obligation has been discharged. Sampling stops as soon as it fires.

The prefix requirement is the constraint that shapes the whole policy design. Both `_parse_stack` and `get_valid_tokens` require a left-to-right prefix of a single expression. This is why the joint autoregressive policy of Section 8.2 permutes at slice granularity only and never reorders tokens within a slice — doing otherwise would invalidate every grammar check at once.

4.4 Exponents, and why the cap matters

`intpower` fits its exponent numerically, then snaps it to a nonzero integer via `_snap_int_power`, clipped to `+INTPOWER_MAX`. The `search`, however, is narrower than the snap suggests: the VARPRO grid is `range(1, INTPOWER_MAX+1)` and the nonlinear fallback bounds exponents to `[0.5, INTPOWER_MAX+0.5]`. **Negative exponents are therefore never explored in practice**, despite the docstring wording.

The cap does not remove high powers from the search space. Because `mul` is *n*-ary and each `intpower` fits its own exponent, x^9 remains expressible as $x^3 * x^3 * x^3$. Verified by direct evaluation: that structure evaluates to 512.0 at $x = 2$, is a complete expression, and is reachable by the sampler. What the cap changes is the *cost*.

Term	Cap = 9: tokens / constants	Cap = 3: tokens / constants
x^2, x^3	2 / 3	2 / 3 (unchanged)
x^5	2 / 3	8 / 6
x^9	2 / 3	8 / 9
x^{12}	8 / 6	14 / 12

Everything cubic or below — Duffing, cubic coupling, every truth in the simulated library — is completely unaffected. Higher powers become three to four times more expensive in tokens and constants, so the search must commit to finding one rather than stumbling into it. A genuine ninth-power term is still discoverable; a noise-driven one is no longer free.

There is a second, less obvious benefit, shown in Figure 5: the cap also shrinks the exponent grid, which moves several important cases back onto the exact fitting path.

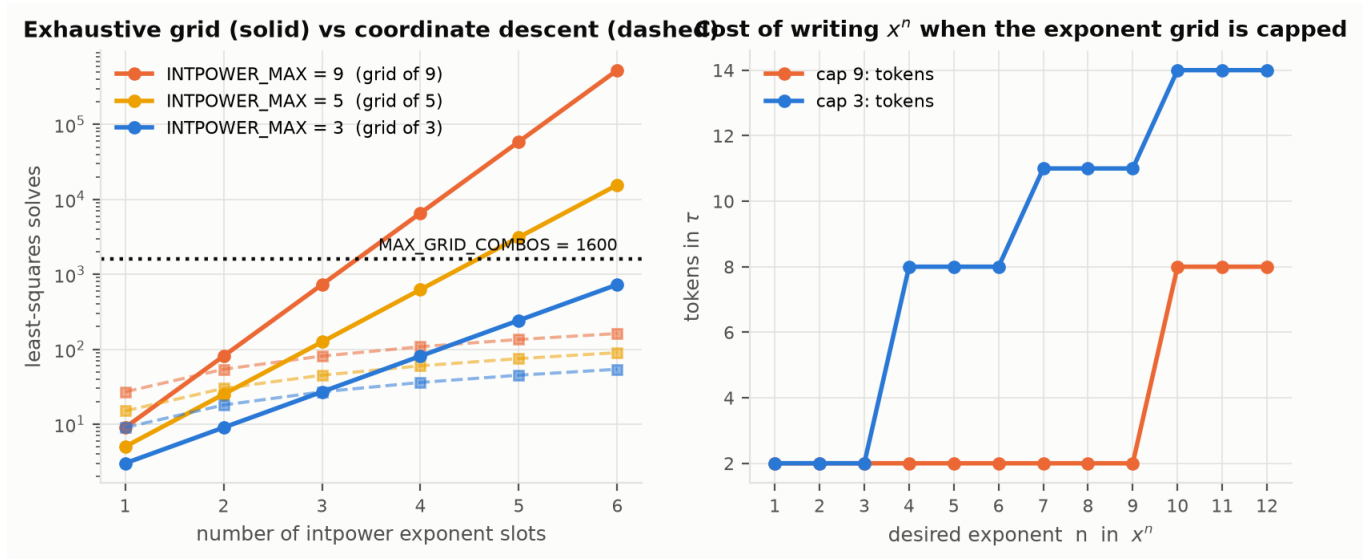


Figure 5. Left: the number of least-squares solves the exponent search needs. Solid lines are the exhaustive product grid, dashed are cyclic coordinate descent; the dotted line is $\text{MAX_GRID_COMBOS} = 1600$, the threshold at which the code switches. With cap 9 a four-slot expression needs 6561 combinations and falls onto coordinate descent, which is not globally optimal; with cap 3 it needs 81 and stays exhaustive. The expanded cube $(q_1 - q_2)^3$ has exactly four slots. Right: the token cost of writing x^n when the exponent grid is capped.

5 Expression evaluation and the four-implementation problem

A token sequence has to be turned into numbers in four different contexts, and `discode_core.py` contains four separate walkers that do it. They are independent code paths over the same grammar.

Function	Produces	Used by
<code>evaluate</code>	A torch tensor, evaluated directly	Torch-side evaluation on normalised features
<code>compile_to_numpy</code>	A compiled Python lambda over numpy arrays	The reward path — one compile, many trajectory evaluations
<code>_build_param_code</code>	A numpy expression string parameterised by a constant vector <code>c</code>	The nonlinear constant fitter, which needs to vary <code>c</code>
<code>expr_to_sympy_str</code>	A sympy-parseable string	Denormalisation to physical units, and pretty printing
<code>_parse_tree + _expand_monomials</code>	A node tree, then a flat monomial list	The VARPRO fast path
<code>expr_to_str</code>	A human-readable string	Console reporting

These implementations must agree exactly on constant ordering, on the coefficient/exponent convention for power operators, and on how n-ary nodes consume their end. A divergence between any two of them produces a silent, self-consistent wrong answer: constants fitted against one interpretation, scored under another. This is the single largest structural risk in the codebase, and it is the reason the joint autoregressive policy was placed in a new module rather than added here — keeping the count at four rather than five.

All four share the same numerical guards. Powers clip to $\pm 1e15$; a negative exponent adds $1e-8$ to the base before dividing; odd integer powers preserve sign via $\text{sign}(u)^{(n \bmod 2)}$ so that x^3 is genuinely odd rather than $|x|^3$; and any non-finite intermediate aborts the evaluation, returning `None` so the candidate is discarded rather than scored on garbage.

5.1 Denormalisation

Everything internal is in normalised units. `denormalize_expr` converts a fitted candidate back to physical units for reporting: it builds the sympy string, substitutes $x_k = (\text{physical} - X_{\text{mean}}[k]) / X_{\text{std}}[k]$, scales by `y_std` and shifts by `y_mean`, then expands. The result is the paste-ready form printed at the end of every run.

```
DISCOVERED_EXPRS = [
    "xddot = -0.5001*x**3 - 1.0*x - 0.3*xdot",
]
```

Because the default normalisation is scale-only, `X_mean` is zero and the substitution is a pure scaling — which is exactly why the expanded polynomial stays as short in physical units as it was in normalised ones.

6 The reward: a per-DOF work-energy balance

6.1 The computation

`energy_reward(exprs, max_traj, horizon, w_acc)` takes one `(tau, consts)` pair per DOF — or `None` for a DOF with no expression yet — and returns a single scalar. For each trajectory and each present DOF it:

- compiles the expression to numpy and evaluates it on the normalised measured states, then converts back to physical acceleration as $y_mean + y_std * pred$;
- forms the cumulative predicted work $cumtrapz(v_d * a_d, t)$ and the measured kinetic-energy change $0.5 * (v_d^2 - v_d(0)^2)$;
- takes the absolute difference, divides by $std(dKE_d)$ when ENERGY_NORMALIZE is on, and averages over time;
- optionally blends in acceleration NRMSE (Section 6.4);

The per-DOF residuals are averaged, then the per-trajectory means are averaged, and finally $r = 1/(1 + \text{mean residual})$. A DOF whose expression is None is omitted from both sides so the balance stays consistent.

The order of operations matters and is easy to get wrong. The reward is *the reward of the mean residual*, $1/(1 + \text{mean_d res_d})$, not the mean of the per-DOF rewards, $\text{mean_d}[1/(1+\text{res_d})]$. The second is always the larger. `discode_score.py` prints per-DOF columns in the second form for diagnosis but the headline number in the first, so that it matches training.

6.2 Why the DOFs are scored in isolation

`energy_worker` scores one DOF at a time, with the other DOFs' expressions set to None. It accepts a `partner-expressions` argument purely for API compatibility and ignores it. The reasoning is worth reproducing in full, because it is the correction of a real defect that produced three separate symptoms.

The per-DOF work-energy relation is an exact kinematic identity. Coupling forces are already inside `a_d`, so DOF `d`'s balance is complete without any knowledge of the others — the constant fitter exploits this too, setting `cum_other = 0`. Averaging a partner's residual into the score therefore adds **no within-epoch information**: it is a monotone shift shared by every candidate of that DOF, so rankings are unchanged. But it makes scores **incomparable across epochs**, because the partner elite changes underneath them.

That single defect caused: an epoch-0 asymmetry, where a candidate scored without a partner keeps the elite slot forever; stale buffer entries driving unfair pruning and `R_alpha` drift; and `best_r` being dethroned purely because the partner improved. Scoring in isolation removes all three at once.

Coupling terms remain fully discoverable. In an N-DOF run every DOF's states are grammar variables, so DOF 0's expression may reference `q2`, `q2dot` and so on. Isolation is about the *scoring*, not the search space.

6.3 What the residual actually looks like

Work-energy balance: predicted work in vs measured work **Normalised running residual (what the reward averages)**

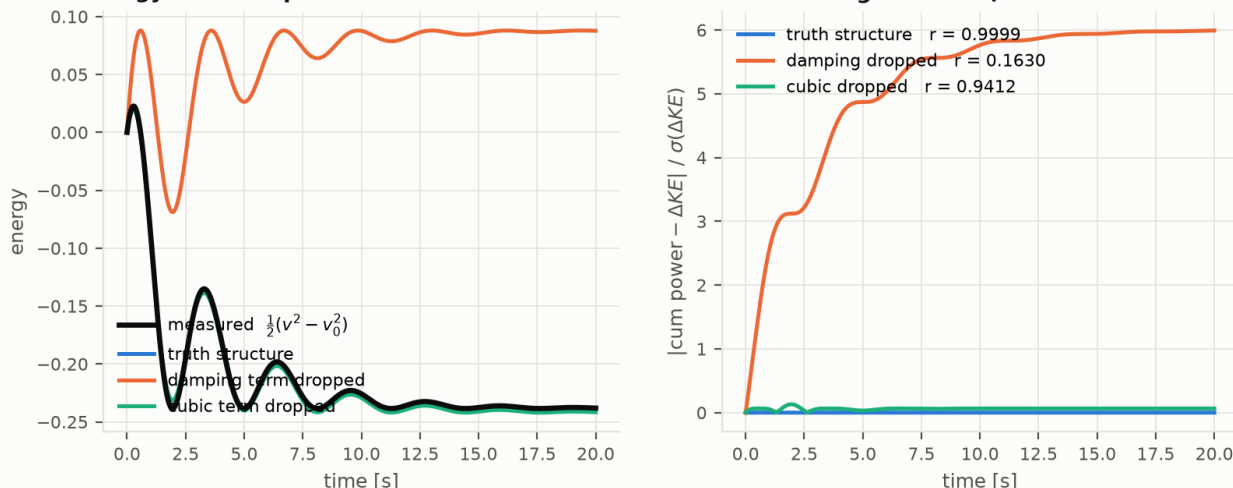


Figure 6. Generated by running the SDOF Duffing truth. Left: the measured kinetic-energy change against the cumulative predicted work for three structures. The truth tracks it exactly; dropping the cubic term produces a visible but modest gap; dropping the damping term diverges steadily, because damping is the only term that does net work over a cycle. Right: the same three as normalised running residuals — this curve, averaged over time, is what the reward penalises. Measured rewards: 0.9999 (truth), 0.9412 (no cubic), 0.1630 (no damping).

6.4 The w_{acc} blend

The pure energy objective has a known weakness, and so does pure acceleration NRMSE. The code therefore blends them:

```
residual = (1 - w_acc) * energy_residual + w_acc * accel_NRMSE
```

Both terms are already dimensionless — energy by $\text{std}(\text{dKE})$, acceleration by $\text{std}(a_{\text{measured}})$ — so the weighted sum is meaningful without further conversion, and w_{acc} is a genuine weight rather than an arbitrary units choice. Crucially both are **linear in the coefficients given the exponents**, so the blend is a single stacked least-squares solve and the closed-form constant fit survives intact.

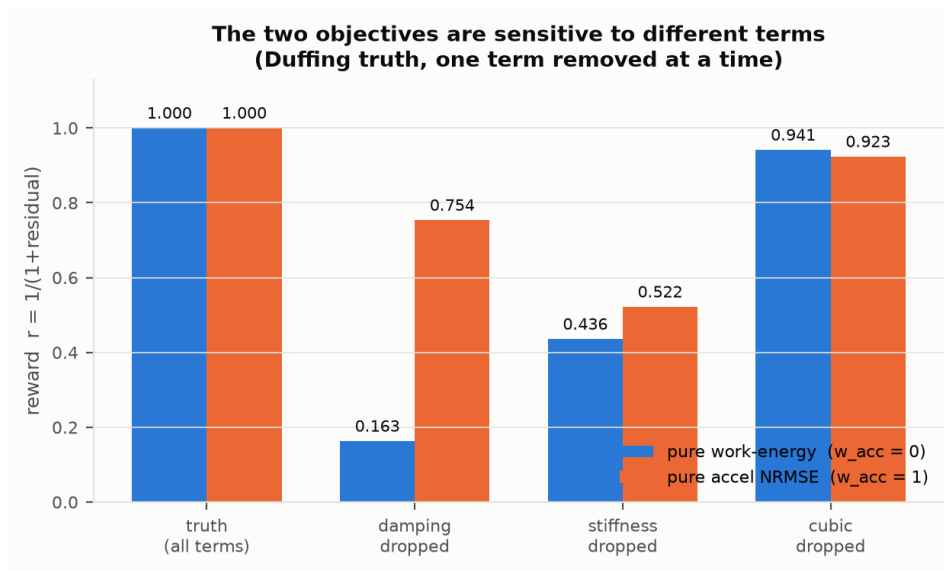


Figure 7. Measured on the SDOF Duffing truth by removing one term at a time and rescored. Dropping damping entirely costs the energy reward 0.9999 $\rightarrow$ 0.1630 but costs pure NRMSE only 1.0000 $\rightarrow$ 0.7535. This is the effect the blend exists to capture: under pure NRMSE a small dissipative term is barely worth selecting.

The measured numbers support the design decision but not every claim made for it. Dropping damping is punished about 4.6x harder by the energy objective than by NRMSE, which is the documented motivation and is clearly real. However, on this system the energy objective is *also* the stricter of the two for stiffness (0.4357 versus 0.5218) — whereas the module docstring, working from a different truth, reports the ordering reversed for stiffness. **The relative sensitivity is system-dependent**; the robust and reproducible claim is the damping one.

The default is $w_{\text{acc}} = 0.5$ in every driver. One practical note: the acceleration block needs no cumulative integration, so $w_{\text{acc}} = 1$ is strictly cheaper per candidate than $w_{\text{acc}} = 0$.

6.5 Two reward scales

The stored reward $r = 1/(1+e)$ is bounded and convenient for gating and reporting, but it compresses badly near $r = 1$ — exactly where a converging search spends its time. A residual improvement from 0.040 to 0.015, a factor of 2.7, is a gap of only 0.024 in r -units, which group normalisation buries.

$\log_{\text{residual_score}}(r) = \log(r) - \log(1-r) = -\log(e)$ maps back to the log-residual view, where a k -fold improvement is worth $\log(k)$ *anywhere* on the scale. The map is monotone, so rankings never change; only

gradient magnitudes do. **Buffer contents, gating and reporting stay in r-units; GRPO advantages use log-residual units.**

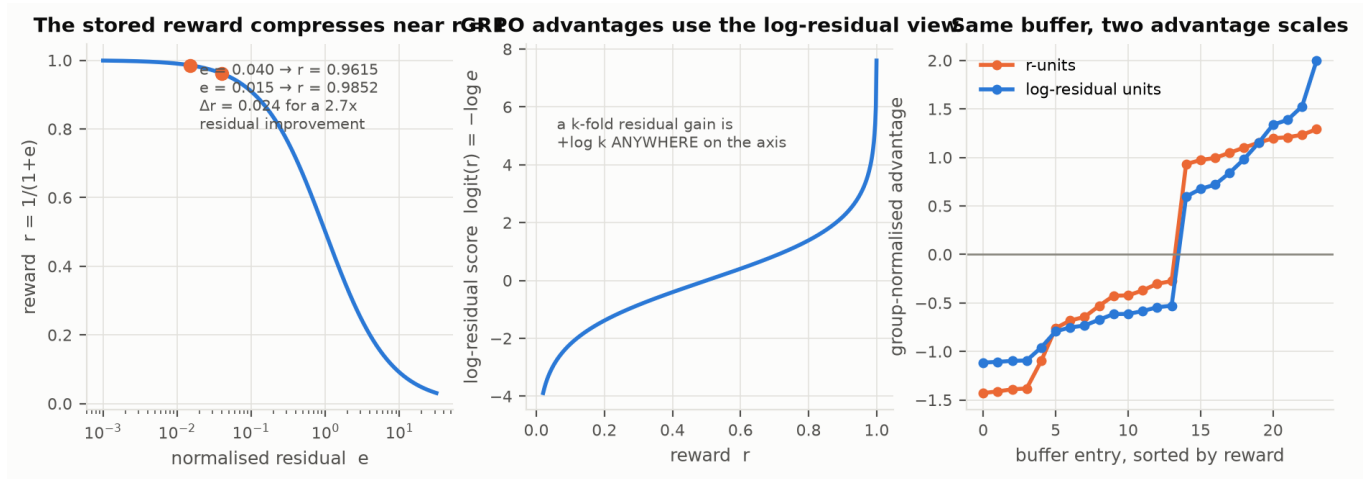


Figure 8. Left: the stored reward compresses near 1. Middle: the log-residual transform used for advantages. Right: the same synthetic buffer normalised in each scale — in r -units the well-converged entries collapse into a single cluster and their differences vanish; in log-residual units late-stage refinement keeps a full-sized gradient.

7 Constant fitting by variable projection

The policy searches over *structure* only. Every free constant is fitted numerically before the candidate is scored. This split is what makes the discrete search tractable, and it means the quality of the fit directly determines whether a good structure is recognised as good.

`optimise_consts_energy(tau, target_dof, horizon)` is the entry point. It tries a closed-form path first and falls back to a general nonlinear optimiser only when it must.

7.1 Why a closed form exists at all

Both halves of the objective are **linear in the coefficients once the exponents are fixed**:

```
energy : (dKE - INT(v*ym)) / s_e  ~=  SUM_j  c_j * ys * INT(v * phi_j) / s_e
accel  : (a_meas - ym)          / s_a  ~=  SUM_j  c_j * ys * phi_j          / s_a
```

where ϕ_j is the j -th monomial basis function with its own amplitude set to 1. This is a classic variable-projection (VARPRO) structure: the linear coefficients are eliminated by least squares, leaving only the nonlinear exponents to search.

The blend is handled by **stacking**: the energy rows are scaled by $\sqrt{1-w}$ and the acceleration rows by $\sqrt{w}$, and both the target vector and the design matrix carry the same weights. Weighting only the target would not weight the problem at all — it would merely rescale the residual — so both must be done together.

7.2 Monomial expansion and its boundary

`_expand_monomials` distributes products over sums into a flat monomial list. It supports bare variables and power operators **whose child is a bare variable**. Anything else — a power of a sum or a product, or a sub node — returns `None`, and the caller falls back to the nonlinear optimiser rather than silently mis-fitting. The expansion also aborts past `MAX_MONOMIALS = 40`.

There is a second requirement that is easy to miss: **separability**. Each monomial must own at least one constant slot that no other monomial uses, so that freezing the rest to 1 reproduces independent amplitudes. If no such exclusive slot exists the structure is not separable and the fast path declines.

7.3 The exponent search

With the coefficients eliminated, only the exponent slots remain. The strategy depends on how many combinations the product grid would need:

Case	Strategy
No exponent slots	One exact least-squares solve. Done.
Product grid $\leq$ <code>MAX_GRID_COMBOS</code> (1600)	Exhaustive sweep over the product grid — globally optimal on the grid . For intpower the grid is the exact integer set, so the fit is exact per combination and needs no polish.
Product grid > 1600	Cyclic coordinate descent: sweep one slot at a time, up to <code>MAX_CD_PASSES = 3</code> passes. Costs $\text{sum}(\text{grid sizes})$ per pass instead of the product. Not guaranteed globally optimal, but enormously better than handing the problem to a nonlinear solver.
More than <code>MAX_EXP_SLOTS</code> (40) slots	Decline; fall back to the nonlinear optimiser.
Continuous exponents present	After the grid search, a bounded least_squares polish refines the continuous slots only.

The coordinate-descent branch was added for a specific, measured failure. The expanded-cube structure (q_1^3 , $q_1^2 q_2$, $q_1 q_2^2$, q_2^3) has **four** intpower slots. Under an older rule that refused more than two slots, it was pushed onto the multi-start nonlinear optimiser and systematically under-fitted: **the same structure scored 0.925 fitted that way and 0.999 fitted exactly**. Structurally wrong two-slot candidates got the exact path and therefore beat the truth on reward — a fitting artefact masquerading as a search result.

This is the sharpest illustration of a general principle in this codebase: **a weakness in the constant fitter shows up as a wrong answer from the search**, not as a fitting warning.

7.4 Two optimisations that matter

Row subsampling. Quadrature accuracy requires integrating on the fine time grid, but a least-squares fit of a few dozen coefficients does not need every integrated row. All cumulative integrals run on the full grid; only the rows handed to `lstsq` are strided down to `MAX_FIT_ROWS = 8000`.

Integrated-column caching. A monomial's column depends only on the exponent slots it actually contains, usually one. Caching per (trajectory, monomial, relevant slot values) turns an 18x18 grid into 18 integrations per single-slot monomial instead of 324.

7.5 The nonlinear fallback

When the fast path declines, `_build_param_code` generates a numpy expression parameterised by the constant vector, and `scipy least_squares` runs from up to three of twelve prepared initialisations (positive and negative coefficient scales set by `std(dKE)`, exponent guesses of 1, 2 and 3, plus ones, zeros and two random draws). Exponent slots are bounded; everything else is free. Non-finite predictions are replaced by a large constant residual so the optimiser walks away from them rather than crashing.

This path is slower by orders of magnitude and less reliable. Candidates that land on it are the ones most likely to be under-fitted and therefore under-ranked.

8 The policy

Two architectures are implemented and share one training loop. The joint autoregressive policy is the default; the original masked-diffusion policy remains available unchanged as `architecture='independent'`.

8.1 V0: the masked-diffusion policy

`DiffusionModel` is a transformer over a length-`max_len` sequence of token indices. Its input is a one-hot encoding plus a sinusoidal positional encoding built in *vocabulary* space and conditioned on a diffusion step `t`; it runs a 2-layer encoder and then a 2-layer decoder **on the same input** (the decoder cross-attends to the encoder output of the identical sequence).

Training uses `forward_diffusion`: `min(t, len)` randomly chosen positions of a known-good sequence are replaced by MASK, and the model is asked to score the original tokens. `t` is drawn uniformly from `1..max_len` at every GRPO step.

Sampling is where the limitation lies. `sample_batch` calls the model *once* on an all-MASK input and draws every token from the resulting position-wise marginals:

```
x_M = torch.full((batch_size, max_len), MASK_TOKEN)
p    = torch.softmax(model(x_M, max_len), dim=-1).cpu()
return [sample_expression(p[b], max_len) for b in range(batch_size)]
```

Every token in a batch element is drawn independently from a fixed (position, token) probability matrix. The network can express "position 4 is often intpower" but never "given position 3 is intpower, position 4 should be x1". All structural coherence in a sampled expression comes from the grammar mask and from whatever the constant fitter manages to make of the result.

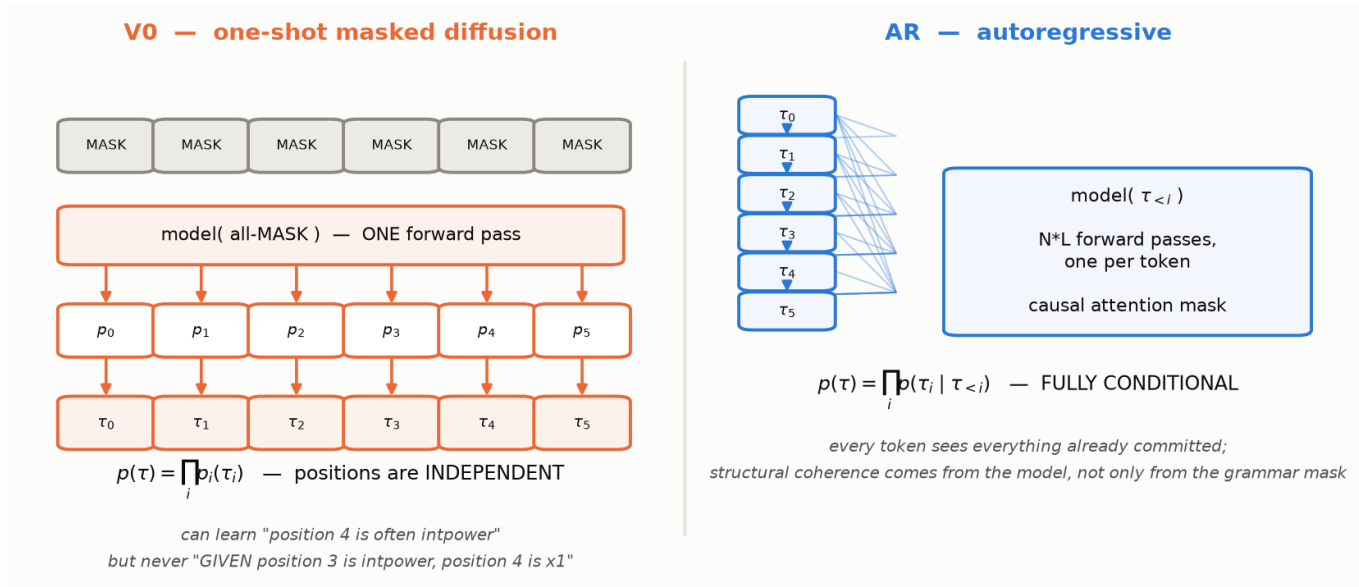


Figure 9. The two factorisations. Left: one forward pass yields independent position-wise marginals, so $p(\tau)$ factorises completely and no token can condition on any other. Right: autoregression conditions each token on the entire committed prefix, at the cost of one forward pass per token.

8.2 The joint autoregressive policy

ARJointPolicy writes every DOF's token sequence as one flattened causal sequence, so a DOF can condition on the expressions already committed for the others. It targets two independent defects, which are worth tracking separately because they have different expected payoffs.

Defect	What it costs	What fixes it
Position factorisation	The policy cannot represent any dependence between token positions. Applies equally to a single-DOF run.	Autoregression. Measurable on SDOF, where there is no jointness to confound it.
Cross-DOF structure is not shared	An internal coupling force appears in two equations at once with opposite sign (Newton's third law). Independent policies must discover the shared subtree twice.	Cross-slice attention. The sign is free because VARPRO fits the leading coefficient, so only the structure must cross.

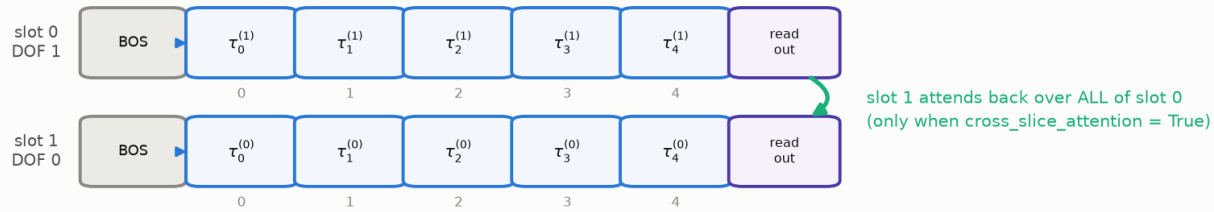
The first is the larger and more certain win. The second is switched off by `cross_slice_attention=False`, which is the only way to attribute a result to one or the other.

Layout

The flattened sequence is `n_dof slices` of `max_len` tokens. A **slot** is a position in that sequence; the DOF occupying it is `pi[slot]`, where `pi` is a permutation that varies across the batch. Under the causal mask, slot `s` sees exactly the slots before it under `pi` — a finite, known set that is stored with the buffer entry as its **context**.

The flattened joint sequence (N = 2 DOF, L = 5 shown)

slice order $\pi = (1, 0) \rightarrow$ slot 0 holds DOF 1, slot 1 holds DOF 0



Teacher forcing is PER SLICE: the input is that slice shifted right by one, starting from its own learned BOS.

- Shifting the flattened sequence instead would feed slot 0's last token into slot 1's first input — leaking across the boundary through the INPUT, where no attention mask can stop it.
- The readout position carries the slice's final token so later slices can attend to it. Without it a token at $l = L-1$ would never be fed in anywhere, and a full-length expression's last token would be invisible to every later slice.
- Permutation happens at SLICE granularity only, never inside a slice — so every slice stays a left-to-right prefix and `get_valid_tokens` / `_parse_stack` / `is_complete`, all of which require a prefix, work untouched.

Four additional input embeddings, each answering a different question:

tok_emb which token	pos_pe where in the slice	dof_emb which DOF this slice IS	slot_emb where in π this slice sits
------------------------	------------------------------	------------------------------------	--

Figure 10. The flattened joint sequence. Each slice occupies $L+1$ input positions: a learned BOS, the L shifted tokens, and a trailing readout position. The three annotations are the design constraints that make the scheme correct.

Two invariants hold the design together, and both are load-bearing:

- **Permutation happens at slice granularity only, never inside a slice.** Every slice stays a left-to-right prefix, so `_parse_stack`, `get_valid_tokens` and `is_complete` — all of which require a prefix — work untouched. This is the single most important constraint in the design.
- **The teacher-forcing shift is per slice, not over the flattened sequence.** Each slice starts from its own learned BOS. Shifting the flattened sequence instead would feed slice $s-1$'s last token into slice s 's first input, so information would cross the boundary through the *input* rather than through attention — and `cross_slice_attention=False` would not actually isolate the slices.

The readout position. A consequence of the per-slice shift is that a slice's token at $l = L-1$ would never be fed in anywhere, so a full-length expression's final token would be invisible to every later slice — undercutting exactly the conditioning the model exists to provide. Each slice therefore gets $L+1$ input positions, the last carrying its final token with its own logits discarded.

Architecture

Three departures from `DiffusionModel`:

- **One causal stack, not encoder-then-decoder.** `n_layers` defaults to 4, the old `n_enc` + `n_dec`, so it is comparable in size to *one* `DiffusionModel`.
- **Conditioning lives in `d_model`, not vocabulary space.** The old sinusoid was built in a `VOCAB_SIZE`-dimensional space — for a 2-DOF grammar that is 10 dimensions, giving two frequency pairs to encode both position and diffusion step, with no room for a third band. Four bands are needed here, so they moved into `d_model`.
- **No diffusion step.** It is meaningless under autoregression, and the `t`-keyed encoding cache goes with it.

```
h[slot s, position l] = tok_emb(x) + pos_pe[l] + dof_emb(pi[s]) + slot_emb(s)
```

tok_emb	which token	dof_emb	which DOF this slice IS
pos_pe	where in the slice	slot_emb	where in π this slice sits

slot_emb is what lets the model distinguish "I am generating first, there is no context" from "I am third, two expressions precede me". dof_emb is necessary because a slice's physical position moves with pi.

Attention masks

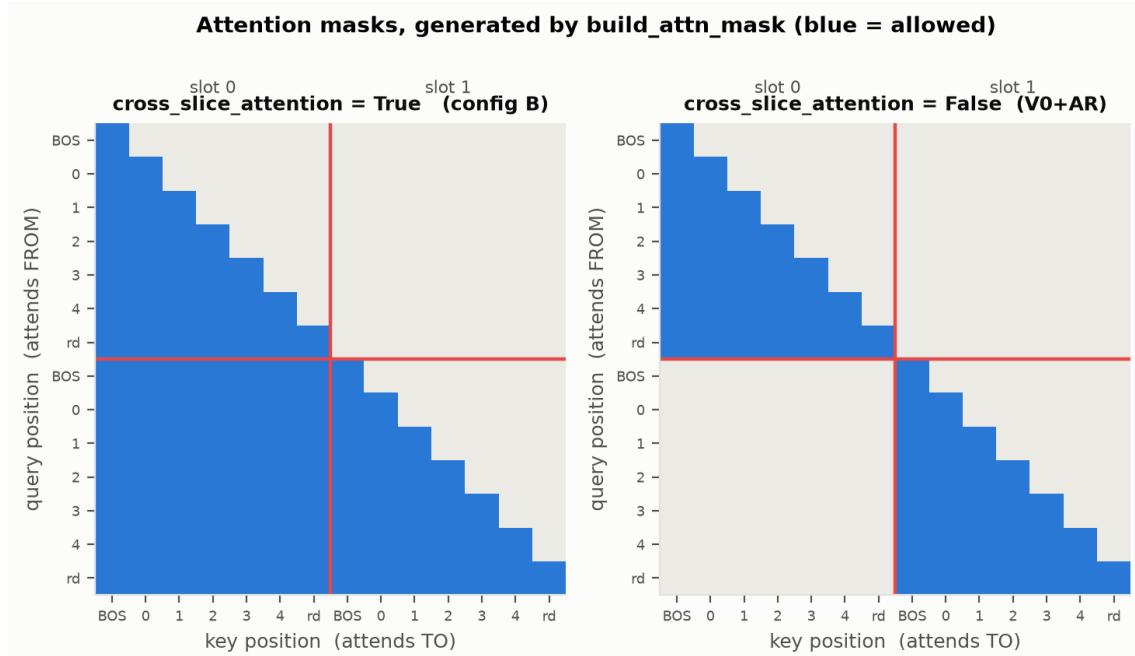


Figure 11. The masks returned by `build_attn_mask`, drawn directly from the function. Blue is allowed. Left: causal across the whole flattened sequence, so slot 1 attends back over all of slot 0. Right: additionally confined to each slice, which yields N independent autoregressive models sharing one set of weights. The two ablation configurations differ by this mask and nothing else.

The mask is cached on $(n_dof, slice_len, flag, device, dtype)$. The diagonal is always allowed, so no row is fully masked and softmax never sees an all-minus-infinity row.

Sampling

`sample_joint_batch` generates the flattened sequence strictly left to right, one forward pass per token, batched over candidates. Three details are easy to get wrong:

- **Validity uses the intra-slice prefix and the intra-slice position.** `get_valid_tokens` computes `remaining = max_len - position`, so passing a flattened index would silently corrupt the length budget.
- **Padding after completion.** When `is_complete` fires at $l = k$, positions $k+1..L-1$ of that slice stay MASK. The update must rebuild exactly this padding, or the train and sample distributions diverge invisibly.
- **Truncation.** When generating slot s , only slices $0..s$ are computed. Causality makes this exact, and it roughly halves the average sequence length.

The dead-end fallbacks from `sample_expression` are reused rather than reimplemented: if the masked distribution sums to zero, fall back to a uniform draw over valid tokens; if that is also empty, the candidate is a genuine dead end and stops incomplete. Incomplete candidates are legitimate and are discarded downstream by the scoring worker.

Beam search

`beam_search_slice` runs a beam **within one slice**, with the preceding slices already committed. A joint beam over all $N \times L$ positions is not worth its branching factor. Unlike the V0 beam, the distribution is recomputed after every committed token — under autoregression there is no static marginal matrix to read rows out of.

8.3 Buffer contexts

A buffer entry is `(energy_reward, tau, consts, context)`, where the context records the slice order, this slice's slot, and **only** the slices that preceded it under `pi`:

```
context = {'dof_order': (1, 0), 'slot': 1, 'prefix_slices': {1: [...]}}
```

Slices that came *after* are irrelevant and must not be stored, or the update would condition on tokens the sample never saw. Storing the context — rather than looking it up live from, say, the current elite of the other DOFs — is what keeps an entry reproducible in isolation. Were the context live, the same entry would mean different things in different epochs, the current, old and reference policies would disagree about what it even is, and cross-epoch comparability would break: the same class of defect that partner-mixing in the reward caused (Section 6.2).

Deduplication keys on tuple(`tau`) alone, never on the context. The same structure found under a different slice order is the same expression, and the reward that gated it was computed in isolation either way.

8.4 Verified invariants

`discode_policy_test.py` contains 27 executable checks. Each exists to catch a specific silent failure — a model with a mis-aligned shift, or with information leaking backwards, trains perfectly happily towards the wrong target.

Invariant	What it catches	Status
Teacher-forcing alignment	An off-by-one in the shift. Overfits one example and checks the argmax reproduces <code>tau[k]</code> , not <code>tau[k+1]</code> .	PASS
Causal masking	Information flowing backwards. Perturbs a token and asserts every logit at or before it is bitwise unchanged.	PASS
Slice isolation	A flattened rather than per-slice BOS shift. With cross-attention off, perturbing any token in slice 0 must leave every logit in slice 1 unchanged — including the last token.	PASS
<code>n_slots</code> truncation	A mask built for the wrong length. Truncated and full forwards must agree bitwise.	PASS
Grammar validity	Any sampled <code>tau</code> violating the grammar. 128/128 sampled expressions from an untrained policy were complete, unique and within <code>max_len</code> .	PASS
Context reproducibility	Batch composition moving an entry's logits, which would make buffer entries incomparable.	PASS
Padding convention	The update building different padding from the sampler.	PASS

9 The training loop and J-GRPO

`DISCODE_TRAIN(system_data, ...)` is the only training loop. State is split in two: one `PolicyState` owning the weights, optimiser, schedule and the old/reference snapshots, and `N DOFState` objects owning the per-DOF replay buffer, gating thresholds, hall of fame and critic. Both architectures expose the same three operations — refresh, sample, update — so the loop body is a single code path.

9.1 One epoch

Step	What happens
0. Refresh	<code>policy_old <- policy</code> every epoch; <code>policy_ref <- policy</code> every <code>G = 5</code> epochs.
1. Sample	The policy emits <code>batch_size</code> candidates per DOF, plus a beam batch every <code>beam_interval</code> epochs. Under the joint architecture a base slice order is chosen for the epoch and then varied across the batch; each candidate carries its context.
2. Score	Every candidate becomes a task (<code>dof</code> , <code>tau</code> , <code>None</code> , <code>horizon</code>) on a <code>ProcessPoolExecutor</code> . The worker fits constants and scores that DOF in isolation, returning (<code>dof</code> , <code>r</code> , <code>tau</code> , <code>consts</code>) or <code>None</code> . Contexts are never sent to the workers — they have no policy and no use for them — and are zipped back by index, which is safe because <code>pool.map</code> preserves order.
3a. Promote	Per DOF: rank by novelty-scaled score, take the top <code>alpha</code> fraction, gate on <code>R_alpha</code> if the buffer is full, deduplicate on <code>tuple(tau)</code> , store the RAW reward, and update the hall of fame.
3b. Critic	Per DOF: a few MSE steps regressing the reward from the token sequence.
3c. Update	One J-GRPO update across every DOF that promoted something. Under the joint architecture this is one backward pass per step with the per-DOF losses summed; under the independent architecture it is the original per-DOF update, unchanged.
3d. Prune	Per DOF: sort by reward, and once the buffer reaches <code>max_buffer</code> drop the worst <code>alpha</code> fraction. Recompute <code>R_alpha</code> .

A DOF that produced no valid candidate sits the epoch out — it does not train on its stale buffer. Under a shared policy this also stops one DOF's stale buffer from pushing the common trunk on its own.

9.2 Buffer promotion in detail

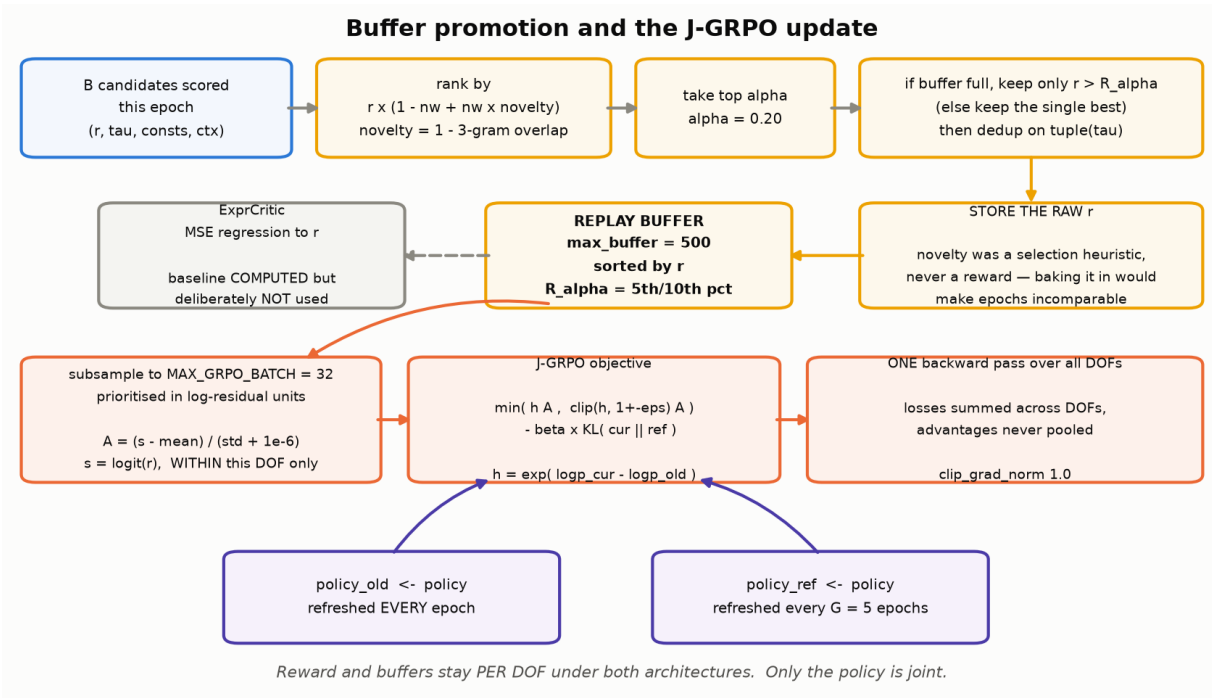


Figure 12. Promotion and the update. Novelty scales the ranking but never the stored value; the buffer is gated on R_alpha and deduplicated on the token sequence; and the advantage is computed within a single DOF’s batch even though the backward pass is shared.

Novelty is a selection heuristic, never a reward. `structural_novelty` returns 1 minus the maximum 3-gram overlap with anything already in the buffer, and candidates are ranked by $r * (1 - nw + nw * novelty)$ with $nw = 0.15$. But the value *stored* is the raw r . Novelty depends on buffer contents at scoring time, so baking it into the stored value would make entries from different epochs incomparable — the same failure mode as partner-mixing in the reward and live buffer contexts. Three separate places in this system have had to learn the same lesson.

Gating. While the buffer has room, the whole top-alpha fraction is promoted. Once it is full, only candidates beating R_alpha (a low percentile of current buffer rewards) are admitted — and if none qualify, the single best is admitted anyway so the buffer never starves. R_alpha is recomputed as the 10th percentile after promotion and the 5th after pruning.

Hall of fame. A separate top-10 list per DOF, never pruned by the buffer policy, so a good early find is not lost to churn.

9.3 The critic is computed and deliberately unused

`ExprCritic` is a small embedding-plus-MLP that regresses the reward from a padded token sequence, trained for a few MSE steps per epoch once the buffer holds at least 16 entries. Its baseline is computed inside the objective and then **not used** for the advantage. The code says so explicitly and keeps the pathway available. Group normalisation over the sampled batch is doing that job instead.

9.4 The J-GRPO objective

The objective is a PPO-clipped policy gradient with a KL anchor to a periodically refreshed reference, computed over buffer entries rather than fresh rollouts.

```

valid = [ e for e in buffer if e.r - R_alpha > 0 ]
if len(valid) > MAX_GRPO_BATCH:      # 32
    prioritise by log-residual advantage, sample without replacement

s = log_residual_score(r)           # logit(r) = -log(e)

```

```

A  = (s - mean(s)) / (std(s) + 1e-6)      # WITHIN this DOF only

h      = exp( logp_cur - logp_old )
obj_k   = min( h * A , clip(h, 1-eps, 1+eps) * A )
total  += obj_k - beta * KL( cur[k] || ref[k] )
entropy += H( cur[k] )

loss = -( total/n + lam_t * entropy/n )

```

Four details are worth calling out.

- **The clip multiplies by A inside the min, not after.** Multiplying after is only equivalent for $A \geq 0$; for $A < 0$ it leaves the ratio unclipped below 1-eps, giving unbounded down-weighting gradients on tokens that are shared with good expressions.
- **Advantages are computed within a single DOF's batch and never pooled.** Reward scales differ wildly between DOFs, and pooling would let the wider-spread DOF dominate a shared trunk.
- **Subsampling is prioritised in log-residual units**, so near-top refinements are not crowded out by the compressed r-scale.
- **Only tokens actually present in the candidate contribute.** Under the AR objective only the positions belonging to that entry's own slot are scored; the prefix slices are context, not targets.

Three degenerate cases. Nothing above R_{α} ; a degenerate advantage spread ($\text{std} < 1e-3$, where normalisation would amplify noise into ± 1 advantages); and zero scored tokens. Under the original per-DOF architecture each of these simply returned and skipped that DOF's update. Under a shared policy that would discard the other DOFs' gradients too, so the joint objective reports "no contribution" for that DOF alone and the step proceeds with the rest.

9.5 Schedules

Schedule	Behaviour
Entropy bonus <code>lam_t</code>	Geometric decay from <code>lam_start = 0.02</code> to <code>lam_end = 0.001</code> across the run. Encourages exploration early and exploitation late.
Learning rate	CosineAnnealingLR from <code>lr = 1e-4</code> down to <code>eta_min = 1e-5</code> over <code>n_epochs</code> .
Reference refresh	Every <code>G = 5</code> epochs. Bounds how far the policy can drift from a fixed anchor.
Trajectory horizon	Currently disabled. <code>get_traj_horizon</code> computes a curriculum fraction rising from 0.30 to 1.0 over the first 20% of epochs, but the function returns <code>t_end</code> before reaching it — the curriculum line is unreachable dead code. Every epoch scores on the full record.

The disabled horizon curriculum is a live discrepancy between the code and its own documentation. It is not obviously a bug — scoring on the full record throughout is defensible and is what every recorded run has actually done — but anyone reading `get_traj_horizon` would reasonably conclude a curriculum is active. It is not.

10 Complete configuration reference

10.1 DISCODE_TRAIN parameters

Parameter	Default	Effect
n_epochs	300	Epochs. Also sets the cosine LR horizon and the entropy decay rate.
batch_size	200	Candidates sampled per DOF per epoch. Dominates scoring cost.
max_len	20	Maximum tokens per expression. Drivers override: 24 for MDOF sim, 40 in the experimental example.
lr	1e-4	Adam learning rate, cosine-annealed to 1e-5.
alpha	0.20	Top fraction promoted per epoch; also the fraction pruned when the buffer is full.
C	5	GRPO steps per epoch.
G	5	Epochs between reference-policy refreshes.
lam_start / lam_end	0.02 / 0.001	Entropy bonus, geometrically decayed.
eps	0.2	PPO clip half-width.
beta	0.01	KL-to-reference weight.
max_buffer	500	Replay buffer cap per DOF.
beam_interval / beam_width	10 / 20	Beam search every N epochs, with this width.
novelty_weight	0.15	Weight of the novelty term in the promotion ranking only.
energy_normalize	True	Divide each DOF residual by std(dKE) so DOFs are comparable.
max_traj	None	Trajectories used by BOTH the fit and the score. None resolves to all.
w_acc	0.5	Blend weight: 0 = pure work-energy, 1 = pure acceleration NRMSE.
use_pool	True	Score candidates on a ProcessPoolExecutor.
center_features	False	Scale-only normalisation. See Section 3.4 before changing.
architecture	'joint'	'joint' or 'independent'.
cross_slice_attention	True	Joint only. False gives N weight-sharing AR models.
n_layers / d_model	4 / 128	Joint policy size.
slice_order	'random'	How slice orders vary across a batch: 'random' mixes permuted and base rows, 'fixed' uses one order.
sample_order	'reward'	The epoch's base order: 'reward' puts the most-converged DOF first, or 'random' / 'fixed'.
seed	None	Slice-order RNG seed.

10.2 Engine constants

Constant	Value	Meaning
POWER_OPS	('intpower',)	Enabled power operators. Add abspower/sgnpower to re-enable continuous exponents.
BINARY_OPS / N_ARY_OPS	['add','mul'] / ['add','sub','mul']	Note 'sub' is in N_ARY_OPS but not in BINARY_OPS, so it never enters the token table.
MAX_TREE_DEPTH	4	Maximum nesting depth.
MIN_EXPR_LEN	6	Minimum tokens before an expression may complete.
INTPOWER_MAX	9	Integer exponent cap. Grid is 1..MAX; negatives are never searched. See Sections 4.4 and 12.3.
MIN_EXP / MAX_EXP	0.5 / 7.0	Continuous exponent bounds (inactive by default).
max_consts	20	Cap on const tokens per expression.
MAX_MONOMIALS	40	Expansion abort threshold for the VARPRO fast path.
MAX_FIT_ROWS	8000	Row budget handed to least squares (integration still runs on the full grid).
MAX_GRID_COMBOS	1600	Above this, the exponent search switches to coordinate descent.
MAX_EXP_SLOTS	40	Above this, the fast path declines entirely.
MAX_CD_PASSES	3	Coordinate-descent sweeps.
MAX_GRP0_BATCH	32	Buffer entries per GRPO step.
HOF_SIZE	10	Hall of fame depth per DOF.
N_WORKERS	min(cpu_count, 16)	Scoring pool size. See Section 12.4 on Windows.
DEVICE	torch.device('cpu')	The whole system runs on CPU.
MAX_ODE_EVALS	50000	Retained for API parity; unused by the energy reward.

11 The ablation design

Two defects were addressed at once, so a single comparison cannot attribute the result. Three configurations of the same entry point separate them.

Label	Configuration	Isolates
V0	architecture='independent '	The original: N one-shot masked-diffusion policies.
V0+AR	architecture='joint', cross_slice_attention=False	Autoregression alone. Both defects' fixes are present in code but only the first is active.
B	architecture='joint', cross_slice_attention=True	Autoregression plus cross-DOF structure sharing.

If B beats V0 but V0+AR does not, the gain is cross-DOF sharing. If V0+AR already captures it, the joint trunk is unnecessary complexity and should revert to N independent AR models.

11.1 Where to measure each defect

Position factorisation is cleanest on a single-DOF system, where there is no jointness to confound it. `discode_sdof_sim.py` with `system_key='duffing'` is the reference case.

Cross-DOF sharing requires a shared subtree worth copying. Linear coupling is a single leaf token — found immediately and far too small to be worth copying — so a joint policy can show no advantage on it. The `cubic_coupled` system exists for this:

$$\begin{aligned} q1'' &= -c1*q1' - k1*q1 - a*(q1 - q2)^3 \\ q2'' &= -c2*q2' - k2*q2 + a*(q1 - q2)^3 \end{aligned} \quad (\text{equal and opposite})$$

The cube expands to four monomials ($q1^3$, $q1^2*q2$, $q1*q2^2$, $q2^3$) that appear in **both** equations, so the shared subtree is genuinely multi-token. The sign flip costs a joint policy nothing, because VARPRO fits each monomial's leading coefficient — only the structure has to cross.

The system was verified representable before use: both truth token sequences are complete, reachable token-by-token through `get_valid_tokens`, and fit to $r = 0.9999$ against a ceiling of 0.9999, recovering coefficients that are exactly opposite between the DOFs (-1.2, +3.599, -3.6, +1.2 against +1.2, -3.6, +3.6, -1.201).

11.2 Parameter counts do not match, and are printed

V0 runs N policies and the joint variants run one, so total parameter counts differ by roughly N. Measured on a 2-DOF run: **1,856,530 trainable parameters across 2 networks for V0, versus 796,169 in 1 network for the joint variants.** Both counts print at startup, and `n_layers / d_model` are exposed so a capacity-matched run can be configured explicitly rather than assumed.

11.3 Cost profile

Measured on a 4-core machine, 3 epochs, 40 candidates per DOF, on the coupled-Duffing system:

Configuration	Sampling	Update	Epoch total
V0	0.27 - 0.40 s	1.6 - 3.6 s	165 - 284 s
V0+AR	0.96 - 1.39 s	2.0 - 4.9 s	120 - 155 s
B	1.11 - 1.75 s	2.0 - 4.9 s	585 - 740 s

Autoregressive sampling costs roughly 3x more wall-clock than the one-shot draw and is still under 1% of an epoch. Energy scoring dominates completely, so the KV-caching contingency contemplated in the design was never needed. The much larger epoch total for configuration B is *also* scoring: the joint policy samples longer, more monomial-rich expressions, and VARPRO costs more per candidate. If long runs feel slow, the lever is scoring throughput or max_len — not the transformer.

These runs are three epochs on one seed and are not an ablation result. They establish that all three code paths execute correctly and give a cost profile. A meaningful comparison needs hundreds of epochs per configuration.

12 Failure modes and diagnostics

These are the failures that actually occur in use, in the order you are likely to meet them.

12.1 A low ceiling makes every downstream number meaningless

This is the dominant practical failure. If the measured acceleration closes only a fraction of its own energy balance, the search has a large residual it can chase with whatever fits the noise. Rankings between candidates can invert near the ceiling, so the search is not merely limited — it is optimising measurement artefacts.

A real record measured during this work produced a worst-trial ceiling of **0.2354**, with a monotone decay from trial 0 to trial 3 and a recovery at trial 4 (0.81 / 0.72 / 0.59 / 0.47 / 0.84 on DOF 0). That pattern is systematic rather than random and points at a per-trial conditioning difference. The remedy is upstream: identical filter passes on every channel.

Diagnostic: every driver prints the ceiling before training. Read it first, every run.

12.2 Negative headroom

`discode_score.py` reports **headroom = ceiling - r_energy**. A negative value means the discovered expression closes the energy balance *better* than the measured acceleration does. That is not a success — it is only possible when the displacement, velocity and acceleration channels are mutually inconsistent, typically because they were filtered a different number of times each. It is a data-integrity warning.

12.3 Discovered equations that cannot be integrated

This one follows directly from the reward's greatest strength. Because scoring evaluates the predicted acceleration on the *measured* states, **nothing in the objective penalises an equation that is unintegrable**. A candidate can score well and still have a finite-time singularity.

A worked case from a real run against the 0.2354 ceiling. The discovered DOF 1 expression contained:

```
... - 4663.0*q1dot**7 + 876.2*q1dot**5 ... + 3.865e+4*q2dot**9 - 5.266e+4*q2dot**8 ...
... and DOF 0 contained -6.276e+8*q1*q2**2 and 8.262e+7*q2**3
```

Coefficients of 10^8 on cubic terms of variables whose magnitude is around 10^{-3} , plus seventh-to-ninth powers of velocity. In the fitted range those terms are order one and cancel against each other; their *derivatives* are enormous, so the ODE is violently stiff. Measured behaviour:

- **Every solver** — LSODA, Radau, RK45, BDF — stops at $t = 0.0757$ s, 1.5% of the record, at every tolerance and step-size setting tried.
- The accepted states never exceed $|\text{state}| = 0.736$ and never go non-finite: this is a convergence failure from stiffness, not an overflow. The $1e4$ blow-up guard never legitimately fires.
- Watching through the singularity at tight tolerance, $q2dot$ goes $0.74 \rightarrow 3.7 \rightarrow 8.7 \rightarrow 11.5 \rightarrow \text{infinity}$ inside a single timestep.
- **Only LSODA combined with the blow-up guard turns this into a crash** (`ValueError: f(a)` and `f(b)` must have different signs), because the terminal event's root bracketing fails on an interpolant fitted across a step where the right-hand side exploded. Radau, RK45 and BDF stop cleanly instead.

The dangerous setting. Loosening to $\text{rtol} = 1e-4$ makes LSODA report `success=True`, `status=0` and return the full 5.112 s — with states of 10^{33} that coast flat for the remaining 98%. A full-length plot of pure numerical garbage that looks like it worked. Do not use it.

Confirmed remedy. Deleting exactly the four high-power velocity terms and changing nothing else takes the same equation from 1.5% to **100% of the record with $\max|\text{state}| = 4.563$** . The grammar-side fix is therefore to cap the

exponent grid: setting `INTPOWER_MAX = 3` prevents a ninth-power term from ever being written directly, at no cost to anything cubic or below (Section 4.4).

But understand what the cap does and does not do. It makes divergent equations *improbable*, not impossible — $x^3 * x^3 * x^3$ is still x^9 and would still blow up if the search paid the token cost to build it. The deeper issue is that the objective has no notion of stability at all. A grammar cap is a mitigation on the input side; it does not teach the reward to care.

12.4 Windows: the scoring pool and the pagefile

Windows has no fork, so `ProcessPoolExecutor` uses `spawn`: each of the up-to-16 workers re-imports the driver module to rebuild its namespace, which reaches `import torch` at module level. The `if __name__ == '__main__':` guard stops training from re-running but not the imports. Sixteen simultaneous torch DLL loads can exceed the pagefile, producing `OSError: [WinError 1455]`.

Remedies in order of preference: raise the Windows pagefile (system-managed, or a fixed 32-64 GB); reduce `N_WORKERS`; or set `use_pool=False` for a serial run.

12.5 Plotting pitfalls

- **Configuration drift.** The plotters default to different trim/downsample settings than the drivers. The equations were fitted on a specific time grid; scoring them on another scores a different problem.
- **Trial choice.** Both plotters default to trial 0. On a record with heterogeneous ceilings, plotting a bad trial makes a good equation look broken.
- **Which figure to trust.** The forward simulation compounds error over time, so a small frequency error diverges visibly even when the structure is right. The energy residual is evaluated on the measured states and is exactly what the search optimised. Trust the residual for "is this equation right"; trust the simulation for "is this usable as a model".

13 Limitations and open problems

Issue	Assessment
No stability term in the objective	The largest open weakness. Section 12.3. Options are a stability screen on promoted candidates (costly: a forward simulation per candidate) or a coefficient-magnitude penalty. Both are real changes to the search, not one-liners.
Four grammar implementations	<code>evaluate</code> , <code>compile_to_numpy</code> , <code>_build_param_code</code> and <code>expr_to_sympy_str</code> must agree exactly. A divergence is silent and self-consistent. There is no cross-implementation test.
The horizon curriculum is dead code	<code>get_traj_horizon</code> returns before its curriculum line. Documented behaviour and actual behaviour differ.
Negative exponents are unreachable	The snap function accepts <code>+INTPOWER_MAX</code> but neither the grid nor the nonlinear bounds ever explore negatives. A $1/x$ term cannot be found.
The critic is trained and unused	Deliberate and documented, but it costs compute every epoch for no effect on the update.
CPU only	<code>DEVICE</code> is hard-coded to CPU. Given that energy scoring dominates by two orders of magnitude, this is a reasonable choice, but it caps policy size.
Ablation not yet run at length	The three configurations are implemented and verified to execute, but the comparison that would justify the joint architecture requires hundreds of epochs per configuration and has not been run.

Issue	Assessment
Parameter counts unmatched across the ablation	V0 has roughly N times the parameters of the joint variants. Both are printed, but a capacity-matched run must be configured deliberately.

Appendix A Worked example, end to end

The SDOF Duffing oscillator, from token sequence to physical equation. Every number below was produced by running the code.

A.1 The system

```
q'' = -c*q' - k*q - a*q^3      c = 0.30,  k = 1.0,  a = 0.50

state layout:  x1 = q,  x2 = qdot
grammar:       ['add', 'mul', 'intpower', 'x1', 'x2', 'const', 'end']  (N_TOKENS = 7)
```

A.2 The token sequence

```
tau = ['add', 'x1', 'x2', 'intpower', 'x1', 'end']

add      open an n-ary sum
x1       c1 * q              <- implicit coefficient
x2       c2 * qdot           <- implicit coefficient
intpower c3 * (...)^n        <- TWO slots: coefficient, exponent
x1       child of intpower, c4 * q
end      close the sum

count_total_consts(tau) = 1 + 1 + 2 + 1 = 5
```

A.3 Sampling this sequence

At each step the policy proposes a distribution, `get_valid_tokens` masks it, and one token is drawn. From Figure 4 (a 2-DOF grammar, same mechanics): at position 0 only operators are legal; inside the sum most tokens are legal; and near the length budget end becomes available while operators drop out.

A.4 Fitting the constants

`_expand_monomials` flattens the tree to three monomials — q , $\dot{q}$ and q^n — each with its own exclusive amplitude slot, so the structure is separable. One exponent slot remains, so the exponent grid is swept exhaustively (9 combinations at `INTPOWER_MAX` = 9, 3 at cap 3) and each is solved by exact least squares against the stacked energy and acceleration rows.

A.5 Scoring

The fitted expression is compiled to numpy, evaluated on the measured states, converted to physical acceleration, and run through the work-energy balance. Measured on this system, with one term removed at a time:

Structure	Energy reward	Accel reward
truth (all terms)	0.9999	1.0000
cubic dropped	0.9412	0.9234
stiffness dropped	0.4357	0.5218
damping dropped	0.1630	0.7535

A.6 The result

An 8-epoch run at batch 60 with the joint autoregressive policy and cross-slice attention disabled (the V0+AR configuration, on a single DOF where there is no jointness) reached the truth's own ceiling:

```
Truth          : xddot = -0.3*xdot - 1*x - 0.5*x^3      r_truth = 1.0000
Expr (physical) : xddot = -0.5001*x**3 - 1.0*x - 0.3*xdot + 6.285e-6
energy_reward   : 0.999964                               (first hit at epoch 2 of 8)
```

Exact structural recovery, with coefficients matching the truth to four significant figures.

Appendix B Function reference

B.1 discode_core.py

Function	Purpose
<code>configure_grammar(n_dof, var_names)</code>	Build the token table, arity map and sympy symbols. Call once, first.
<code>set_problem_data(...)</code>	Install normalisation stats, raw trajectories, and the energy/blend options.
<code>tok2idx / idx2tok</code>	Token/index conversion against ALL_TOKENS.
<code>count_total_consts(tau)</code>	Constant slots in pre-order.
<code>_parse_stack(tau)</code>	Parse a prefix into open obligations; returns None on a grammar violation.
<code>get_valid_tokens(tau, position, max_len)</code>	The grammar mask. Section 4.3.
<code>is_complete(tau)</code>	Whether every obligation is discharged.
<code>sample_expression(p, max_len)</code>	Masked draw from a static (position, token) matrix.
<code>sample_batch(model, B, max_len)</code>	V0 sampling: one forward pass, B independent draws.
<code>beam_search_expressions(...)</code>	V0 beam search over the static marginals.
<code>evaluate(tau, x, consts)</code>	Torch evaluation on normalised features.
<code>compile_to_numpy(tau, consts)</code>	Compile to a numpy lambda. Used by the reward.
<code>_build_param_code(tau)</code>	Numpy expression parameterised by the constant vector, for the nonlinear fitter.
<code>expr_to_str / expr_to_sympy_str</code>	Human-readable and sympy-parseable forms.
<code>sympy_to_tau(expr)</code>	Inverse direction: a sympy expression back to a token sequence.
<code>energy_reward(exprs, ...)</code>	The reward. Section 6.1.
<code>log_residual_score(r)</code>	logit(r), the advantage scale.
<code>_energy_contexts(...)</code>	Per-trajectory precomputation shared by both fitters; sets <code>cum_other = 0</code> .
<code>_parse_tree / _expand_monomials</code>	Tree parse and monomial expansion for VARPRO.
<code>_fit_consts_linear(...)</code>	The closed-form / VARPRO fast path.
<code>optimise_consts_energy(...)</code>	Entry point: fast path, then nonlinear fallback.
<code>structural_novelty(tau, buffer)</code>	1 minus the maximum 3-gram overlap.
<code>get_traj_horizon(...)</code>	Returns <code>t_end</code> . The curriculum is unreachable.
<code>init_energy_worker / energy_worker</code>	Pool initialiser and per-candidate scoring task.
<code>forward_diffusion(...)</code>	V0 training corruption: mask <code>min(t, len)</code> random positions.
<code>compute_jgrpo(...)</code>	The V0 objective.
<code>denormalize_expr(tau, consts, dof)</code>	Back to physical units for reporting.

Function	Purpose
<code>DiffusionModel / ExprCritic</code>	The V0 policy and the reward critic.

B.2 `discode_policy.py`

Function	Purpose
<code>ARJointPolicy</code>	The joint causal transformer. Section 8.2.
<code>sinusoidal(max_len, d_model)</code>	Intra-slice positional encoding, in <code>d_model</code> .
<code>build_attn_mask(...)</code>	Causal, optionally slice-confined. Cached.
<code>choose_order(best_r, mode, rng)</code>	The epoch's base slice order.
<code>make_batch_orders(...)</code>	Per-row slice orders: a mixture of permuted and base rows.
<code>make_context(order, slot, slice_taus)</code>	Build the buffer context from the preceding slices only.
<code>sample_joint_batch(...)</code>	Left-to-right joint sampling with grammar masking.
<code>beam_search_slice(...)</code>	Beam within one slice, prefix committed.
<code>beam_candidates(...)</code>	One beam-searched slice per DOF, with contexts.
<code>build_ar_inputs(entries, ...)</code>	Teacher-forcing tensors reproducing the sampler's padding exactly.
<code>jgrpo_ar(...)</code>	The AR objective. Returns (objective, entropy, contributed).

B.3 `discode_data.py` and `discode_analysis.py`

Function	Purpose
<code>SystemData</code>	The universal data container.
<code>load_mat_data(...)</code>	MATLAB v7.3 and v5/v6 loading, trim and downsample.
<code>select_dof(system, i)</code>	Slice a multi-DOF record down to one channel.
<code>TruthSystem / build_truth_system(...)</code>	Declarative simulated-system spec and its integrator.
<code>identity_ceiling(system)</code>	The ceiling diagnostic. Section 3.5.
<code>print_truth_rewards(system)</code>	Score the known structure with the training reward.
<code>generate_dataset(system, ...)</code>	Normalisation stats and raw trajectories.
<code>clean_exprs / build_rhs</code>	Parse pasted expressions into a callable right-hand side.
<code>forward_simulate(...)</code>	Integrate a discovered system, guarded by a terminal blow-up event.
<code>energy_residual / trainer_reward</code>	Reproduce the training residual and reward post hoc.
<code>plot_discovered(...)</code>	The two comparison figures.
<code>score_system(...)</code>	Reward, ceiling and per-trial rows.

All measurements in this report were produced by executing the code in the repository at the time of writing. Figures 2, 4, 5, 6, 7, 8 and 11 are generated directly from the functions they document; Figures 1, 3, 9, 10 and 12 are schematics drawn to match the implementation.